

Student Schedule Management System

Project Overview and Technical Report

Object-Oriented Programming Course Project

CS Group 11

South China Normal University

May 2026

Abstract

This report presents the design and implementation of a web-based student schedule management system developed as a university Object-Oriented Programming course project. The system provides schedule CRUD operations, real-time course queries, automatic schedule fetching from the SCNU academic portal, and an integrated knowledge workspace featuring note upload, semantic search, RAG-based Q&A, and knowledge graph visualization. The backend is built with Python/FastAPI and employs four hand-implemented data structures (hash table, doubly linked list, binary search tree, and queue) as required by the course. The frontend uses Vue2 with plain HTML/CSS/JS. Persistence is file-based (JSON and SQLite), and the knowledge module leverages vector embeddings via mem0/Qdrant with a DeepSeek LLM for summarization and question answering.

Contents

1	Introduction	2
1.1	Motivation	2
1.2	Course Requirements	2
1.3	Technology Stack	2
2	System Architecture	2
2.1	Directory Layout	3
2.2	Request Lifecycle	3
3	Custom Data Structures	4
3.1	Hash Table (Chaining)	4
3.2	Doubly Linked List	4
3.3	Binary Search Tree	4
3.4	Queue	5
4	Data Models	5
4.1	Core Schedule Models	5
4.2	User and Authentication Models	6
4.3	Knowledge Module Models	6

5	Storage Layer	6
5.1	File I/O Utilities	6
5.2	UserStore	6
5.3	ScheduleStore	6
5.4	NoteStore	7
5.5	KnowledgeTreeStore	7
6	Service Layer	7
6.1	AuthService	7
6.2	ScheduleService	7
6.3	SCNUScraper	7
6.4	NoteService	8
6.5	KnowledgeService	8
7	API Design	8
7.1	Authentication (/auth)	8
7.2	Schedule Management (/schedule)	9
7.3	Query (/query)	9
7.4	Notes (/note)	9
7.5	Knowledge (/knowledge)	9
8	Frontend	10
9	SCNU Academic Portal Integration	10
9.1	Primary Path: Reverse-Engineered API	10
9.2	Fallback Path: Playwright Browser Automation	11
9.3	Tertiary Path: PDF Upload	11
10	Knowledge Module Architecture	11
10.1	Document Processing Pipeline	11
10.2	Vector Storage	11
10.3	RAG (Retrieval-Augmented Generation)	12
10.4	Knowledge Graph	12
10.5	Topic Tree	12
11	Testing	12
11.1	Test Categories	13
11.2	Testing Strategy	13
11.3	Running Tests	13
12	Logging and Observability	14
13	Deployment and Configuration	14
13.1	Configuration	14
13.2	Running the Application	14
13.3	Dependencies	14
14	Design Decisions and Trade-offs	15

15 Project Highlights	15
15.1 Solving a Real Problem with Real Data	15
15.2 Reverse Engineering and Web Scraping	16
15.3 Security Considerations	17
15.4 Frontend–Backend Separation	17
15.5 Note Knowledge Clustering and AI-Assisted Learning	17
15.6 Professional Version Control Practices	18
15.7 Production-Grade Engineering Practices	18
16 Conclusion	19

1 Introduction

1.1 Motivation

University students frequently need to check their class schedules—what course is happening now, what courses are today, and what the week looks like. Existing solutions (paper timetables, screenshots, or generic calendar apps) lack integration with the academic system and offer no intelligent features such as note management or knowledge retrieval.

This project builds a purpose-built web application that:

1. Manages semester schedules with full CRUD support.
2. Queries the current, daily, and weekly courses in real time.
3. Fetches schedules directly from the SCNU academic portal (Qiangzhi system).
4. Provides a knowledge workspace where students upload course notes, search semantically, ask questions via RAG, and explore a knowledge graph.

1.2 Course Requirements

The OOP course mandates the hand-implementation of at least four classic data structures without relying on built-in library equivalents. Our system integrates:

- **Hash Table** (chaining) — session store and user index.
- **Doubly Linked List** — ordered per-user course list.
- **Binary Search Tree** — course time index for $O(\log n)$ lookup.
- **Queue** — asynchronous task scheduling for the SCNU scraper.

1.3 Technology Stack

Table 1: Technology stack summary

Layer	Technology
Backend framework	Python 3.11+, FastAPI, Uvicorn
Frontend	HTML/CSS/JS, Vue 2 (CDN)
Persistence	JSON files, SQLite (notes), Qdrant (vectors)
Authentication	Cookie/Session (PBKDF2 or bcrypt)
Vector search	mem0 + Qdrant (local disk mode)
Embedding model	paraphrase-multilingual-MiniLM-L12-v2
LLM	DeepSeek Chat (OpenAI-compatible API)
Testing	pytest (unit + integration + browser E2E)
Version control	Git + GitHub

2 System Architecture

The system follows a layered architecture with clear separation of concerns:

```

Browser (Vue 2 SPA pages)
|
| HTTP (JSON / multipart)

```

```

    v
Routers (thin layer: auth, schedule, query, note, knowledge)
    |
    v
Services (business logic: AuthService, ScheduleService,
          NoteService, KnowledgeService, SCNUScraper)
    |
    v
Storage (file_io, UserStore, ScheduleStore, NoteStore,
         KnowledgeTreeStore, TopicVectorStore)
    |
    v
Persistence (JSON files, SQLite, Qdrant vector DB, disk files)

```

2.1 Directory Layout

```

OOP_project/
+-- app/
|   +-- main.py           # FastAPI entry point
|   +-- config.py         # Configuration constants
|   +-- logging_config.py # Structured logging (Loguru)
|   +-- core/             # Hand-implemented data structures
|       +-- hash_table.py
|       +-- doubly_linked_list.py
|       +-- bst.py
|       +-- queue.py
|   +-- models/           # Pydantic data models
|   +-- storage/          # Persistence layer
|   +-- services/         # Business logic
|   +-- routers/          # API route handlers
+-- static/              # Frontend pages and assets
+-- data/                # Runtime data (gitignored)
+-- tests/               # pytest test suite (~3000 LOC)
+-- docs/                # Design documents
+-- requirements.txt

```

Listing 1: Project directory structure

2.2 Request Lifecycle

A typical authenticated request follows this path:

1. The browser sends a request (e.g., `GET /query/overview`) with the `session_token` cookie.
2. The router extracts the cookie and calls `AuthService.get_student_id(token)`.
3. `AuthService` looks up the token in the in-memory `HashTable<token, SessionRecord>`; returns the student ID or raises `PermissionError`.
4. The router delegates to the appropriate service method.
5. The service loads data from storage, applies business logic, and returns a Pydantic model.

6. FastAPI serializes the model to JSON and sends the HTTP response.

3 Custom Data Structures

All four data structures are implemented from scratch in `app/core/` without using Python's built-in `dict`, `list` (as a linked structure), or `queue` module equivalents.

3.1 Hash Table (Chaining)

File: `app/core/hash_table.py` **Lines:** 222

Design:

- Generic class `HashTable[K, V]` with configurable initial bucket count and load factor (default 0.75).
- Each bucket is a circular singly linked list (`_Bucket`) with a tail pointer for $O(1)$ append.
- Custom hash function: for integers, masks to 31 bits; for other types, applies a multiplicative hash (factor 131) over the UTF-8 `repr` bytes.
- Automatic resizing (double capacity) when load factor is exceeded.
- Supports Python dunder protocols: `__getitem__`, `__setitem__`, `__delitem__`, `__contains__`, `__len__`, `__iter__`.

Usage in the system:

- `AuthService._sessions`: maps session tokens to `SessionRecord` for $O(1)$ authentication on every request.
- `UserStore._users`: maps student IDs to `User` objects for fast lookup.
- `ScheduleService._tasks`: maps task IDs to `FetchTaskStatus` for async scraper tracking.
- `ScheduleService._build_week_courses`: temporary hash table to group courses by weekday.

Complexity: Average $O(1)$ for `set`, `get`, `delete`; worst-case $O(n)$ under pathological hash collisions.

3.2 Doubly Linked List

File: `app/core/doubly_linked_list.py` **Lines:** 158

Design:

- Each `Node` holds arbitrary data plus `prev/next` pointers.
- Maintains `head`, `tail`, and `_size`.
- Operations: `append`, `prepend`, `insert_after`, `remove`, `find`, `find_by` (predicate), `to_list`, `from_list`, `clear`.
- Supports Python iteration via `__iter__`.

Usage: Stores each user's course list ordered by (`weekday`, `period_start`). Enables efficient insertion/deletion when courses are added or removed, and sequential traversal for "today's courses" queries.

Complexity: $O(1)$ append/prepend, $O(n)$ search/remove by value.

3.3 Binary Search Tree

File: `app/core/bst.py` **Lines:** 61

Design:

- Generic `BinarySearchTree[K, V]` with recursive `insert`, `search`, and `delete`.
- Deletion uses in-order successor replacement (find minimum in right subtree).
- Keys must support comparison operators.

Usage: Course time index where `key = weekday * 100 + period_start`. The “query current course” feature computes the current key and performs a BST search to find the matching course in $O(\log n)$ average time.

Complexity: Average $O(\log n)$ for insert/search/delete; worst-case $O(n)$ for degenerate (sorted) input.

3.4 Queue

File: `app/core/queue.py` **Lines:** 55

Design:

- Linked-list-based FIFO queue with `head` and `tail` pointers.
- Operations: `enqueue`, `dequeue` (raises `IndexError` if empty), `peek`, `is_empty`, `size`.

Usage: The SCNU scraper task queue. When a user requests schedule fetching, the task is enqueued; a background thread dequeues and processes tasks serially to avoid overwhelming the academic portal with concurrent requests.

Complexity: $O(1)$ enqueue and dequeue.

4 Data Models

All models are defined using Pydantic `BaseModel` for automatic validation and JSON serialization.

4.1 Core Schedule Models

Table 2: Course model fields

Field	Type	Description
<code>id</code>	<code>str</code>	UUID primary key
<code>name</code>	<code>str</code>	Course name
<code>teacher</code>	<code>str</code>	Instructor name
<code>location</code>	<code>str</code>	Classroom
<code>weekday</code>	<code>int</code>	1 (Mon) – 7 (Sun)
<code>period_start</code>	<code>int</code>	Starting period (1–12)
<code>period_end</code>	<code>int</code>	Ending period (1–12)
<code>weeks</code>	<code>list[int]</code>	Active week numbers
<code>week_type</code>	<code>Literal</code>	"all" / "odd" / "even"

The `Schedule` model wraps a list of courses with semester metadata (`student_id`, `semester`, `semester_start`).

4.2 User and Authentication Models

- **User:** `student_id`, `name`, `password_hash`, `scnu_account`.
- **UserCreate:** registration payload with validation constraints.
- **UserLogin:** login payload.
- **UserInfo:** public-facing user info (no password hash).

4.3 Knowledge Module Models

- **Note:** metadata for an uploaded document (`id`, `student_id`, `course_id`, `filename`, `file_type`, `title`, `summary`, `chunk_count`, `timestamps`).
- **NoteChunk:** a text segment extracted from a note (`chunk_id`, `note_id`, `heading`, `content`, `chunk_index`).
- **KnowledgeTopic:** a node in the knowledge tree (`id`, `name`, `parent_id`, `child_ids`, `note_ids`, `summary`, `keywords`).
- **KnowledgeTree:** the full tree structure per student per course.
- **GraphNode/GraphLink/GraphResponse:** D3-compatible graph data for the knowledge graph visualization.
- **SearchResult, AskResponse:** semantic search and RAG Q&A response models.

5 Storage Layer

The storage layer abstracts persistence behind clean interfaces, allowing the service layer to remain agnostic to the underlying storage mechanism.

5.1 File I/O Utilities

`app/storage/file_io.py` provides:

- **Atomic writes:** writes to a temporary file, then atomically replaces the target via `os.replace`. Includes a retry mechanism for Windows permission issues.
- **Safe reads:** returns a deep-copied default if the file does not exist; raises on malformed JSON.
- **Model serialization:** `model_to_dict()` handles both Pydantic v1 and v2 models.

5.2 UserStore

- Persists all users to `data/users.json`.
- Maintains an in-memory `HashTable<student_id, User>` index for $O(1)$ lookups.
- On startup, loads all users from disk into the hash table.
- Thread-safe via `threading.Lock`.

5.3 ScheduleStore

- Each student's schedule is stored as `data/schedules/{student_id}.json`.
- Courses are always sorted by (`weekday`, `period_start`, `period_end`, `name`, `id`) before saving.
- Provides `initialize`, `replace`, `add_course`, `update_course`, `delete_course`.
- Thread-safe via `threading.Lock`.

5.4 NoteStore

- Uses SQLite (`data/notes.db`) for note metadata and chunk storage.
- Provides CRUD for notes and chunks, with filtering by student and course.
- Original uploaded files are stored on disk at `data/note_files/{note_id}.{ext}`.

5.5 KnowledgeTreeStore

- Persists the hierarchical topic tree as JSON files per student/course.
- Supports load, save, and deletion operations.

6 Service Layer

6.1 AuthService

Handles user registration, login, logout, and session validation.

- **Password hashing:** prefers bcrypt via `passlib`; falls back to PBKDF2-SHA256 (390,000 iterations) if bcrypt is unavailable.
- **Session management:** generates `secrets.token_urlsafe(32)` tokens; stores `SessionRecord(s, expires_at)` in the custom `HashTable`.
- **Session expiry:** configurable (default 7 days); expired sessions are lazily evicted on access.
- **Thread safety:** all session mutations are protected by a `threading.Lock`.

6.2 ScheduleService

Core business logic for schedule management:

- **Semester initialization:** validates ISO date format, creates or updates schedule metadata.
- **Course CRUD:** validates period ranges and week numbers before delegating to storage.
- **Time-based queries:** computes current week number from `semester_start`, determines current period from the system clock against a 12-period timetable, and filters courses by weekday/week/period.
- **Dashboard overview:** aggregates user info, current course, today's courses, and weekly timetable into a single response to minimize frontend round-trips.
- **SCNU fetch:** submits scraping tasks to a background thread; tracks task status in a `HashTable`.

6.3 SCNUScraper

Fetches schedules from the SCNU academic portal (`jwxt.scnu.edu.cn`):

- **Primary path (reverse-engineered API):** obtains a CSRF token and RSA public key from the login page, encrypts the password with RSA, authenticates via POST, then queries the schedule endpoint for all weeks of the semester.
- **Fallback path (Playwright):** uses browser automation to log in via the SSO portal, captures session cookies, then calls the schedule API.
- **PDF parsing:** uses `pdfplumber` to extract tabular schedule data from downloaded PDF timetables as a tertiary fallback.

- **Rate limiting:** tasks are processed serially via the queue to avoid IP blocking.

6.4 NoteService

Handles the note upload pipeline:

1. Saves the raw file to disk.
2. Extracts text using `pdfplumber` (PDF) or `python-docx` (DOCX).
3. Splits text into chunks using heading-aware segmentation:
 - Regex patterns detect headings (Markdown #, Chinese chapter markers, numbered sections).
 - Long sections are split by paragraph boundaries or hard-truncated at 500 characters with 50-character overlap.
4. Stores note metadata and chunks in the NoteStore.

6.5 KnowledgeService

The most complex service, providing:

- **Vector indexing:** uses `mem0` (backed by Qdrant in local disk mode) with the `paraphrase-multilingual-MiniLM-L12-v2` embedding model (384 dimensions).
- **Semantic search:** queries the vector store; falls back to lexical (token overlap) search if the vector store is unavailable.
- **RAG Q&A:** retrieves top-5 relevant chunks, assembles a prompt, and calls the DeepSeek Chat API. Gracefully degrades to showing raw search results if the LLM is unavailable.
- **Knowledge tree management:** CRUD for hierarchical topics with parent/child relationships, note assignment, and automatic topic routing.
- **Topic vector store:** maintains a separate vector index for topics to enable query-to-topic routing.
- **Knowledge graph construction:** builds a force-directed graph by computing pairwise chunk similarities (vector-based or lexical fallback), filtering edges by a minimum score threshold.
- **LLM summarization:** generates titles and summaries for uploaded notes via DeepSeek.

7 API Design

The system exposes a RESTful API organized into five router modules. All protected endpoints require a valid `session_token` cookie.

7.1 Authentication (/auth)

Method	Endpoint	Description
POST	<code>/auth/register</code>	Register a new local account (201)
POST	<code>/auth/login</code>	Login; sets <code>session_token</code> cookie (200)
POST	<code>/auth/logout</code>	Clears session and cookie (204)

Method	Endpoint	Description
GET	/auth/me	Returns current user info (200/401)

7.2 Schedule Management (/schedule)

Method	Endpoint	Description
GET	/schedule	Get full schedule
POST	/schedule	Initialize/update semester metadata
POST	/schedule/upload	Upload JSON or PDF schedule file
POST	/schedule/fetch	Submit async SCNU scraping task (202)
GET	/schedule/fetch/{id}	Poll scraping task status
POST	/schedule/course	Add a course manually (201)
PUT	/schedule/course/{id}	Update a course
DELETE	/schedule/course/{id}	Delete a course (204)

7.3 Query (/query)

Method	Endpoint	Description
GET	/query/overview	Dashboard overview (user + schedule + current/today/week)
GET	/query/now	Current course (or null)
GET	/query/today	Today's courses
GET	/query/week	This week's timetable
GET	/query/week/{offset}	Offset week timetable

7.4 Notes (/note)

Method	Endpoint	Description
POST	/note/upload	Upload PDF/DOCX; returns note + chunks (201)
GET	/note/list	List notes (filterable by course)
GET	/note/{id}	Get note detail with chunks
PUT	/note/{id}	Update note metadata
GET	/note/{id}/file	Download original file
DELETE	/note/{id}	Delete note and vectors (204)

7.5 Knowledge (/knowledge)

Method	Endpoint	Description
POST	/knowledge/search	Semantic chunk search
POST	/knowledge/ask	RAG Q&A
GET	/knowledge/tree	Get knowledge topic tree
POST	/knowledge/tree/topic	Create topic
PUT	/knowledge/tree/topic/{id}	Update topic
DELETE	/knowledge/tree/topic/{id}	Delete topic
POST	/knowledge/tree/topic/{id}/assign	Assign note to topic
DELETE	/knowledge/tree/topic/{id}/assign	Unassign note
GET	/knowledge/graph	Build and return knowledge graph

8 Frontend

The frontend consists of three main pages served as static HTML with Vue 2 for reactivity:

1. **Login Page** (/login → static/index.html): registration and login forms. Redirects to dashboard on success.
2. **Dashboard** (/dashboard → static/dashboard.html): displays current course, today's courses, weekly timetable grid (7 columns × 12 periods), semester initialization, file import, SCNU fetch, and course CRUD.
3. **Knowledge Workspace** (/knowledge-workspace → static/knowledge_workspace.html): per-course workspace with note upload, topic tree management, document preview (mammoth.js for DOCX), semantic search, RAG Q&A, and knowledge graph visualization (Cytoscape.js).

Key frontend libraries (loaded from node_modules via static mounts):

- Vue 2 — reactive data binding and component logic.
- Cytoscape.js — knowledge graph rendering (force-directed layout).
- Mammoth.js — DOCX-to-HTML conversion for in-browser preview.

Authentication flow: all `fetch()` calls include `credentials: "include"` to send the session cookie. Protected pages are server-guarded: the backend returns a redirect to /login if the cookie is missing or invalid.

9 SCNU Academic Portal Integration

The system integrates with the SCNU academic portal (`jwxt.scnu.edu.cn`) to automatically import student schedules.

9.1 Primary Path: Reverse-Engineered API

1. **GET login page:** extract CSRF token from HTML.
2. **GET public key:** retrieve RSA modulus and exponent from `/xtgl/login_getPublicKey.html`.
3. **Encrypt password:** RSA-encrypt the plaintext password using the retrieved public key.
4. **POST login:** submit student ID, encrypted password, and CSRF token.
5. **GET schedule:** query `/kbcx/xskbcx_cxXsgrkb.html` with semester and week parameters.

6. **Parse response:** extract course fields (name, teacher, location, periods, weeks) from the JSON response and normalize into `Course` objects.

9.2 Fallback Path: Playwright Browser Automation

When the reverse-engineered API fails (e.g., due to portal updates), the system falls back to Playwright:

1. Launch a headless Chromium browser.
2. Navigate to the SSO authentication URL.
3. Fill in credentials and submit the login form.
4. Wait for redirect to the academic portal.
5. Extract session cookies from the browser context.
6. Use the cookies to call the schedule API via `requests`.

9.3 Tertiary Path: PDF Upload

Users can manually download their schedule PDF from the portal and upload it. The system uses `pdfplumber` to extract the timetable grid and regex patterns to parse course information from cell text.

10 Knowledge Module Architecture

The knowledge module extends the schedule system into a learning tool. Its architecture combines document processing, vector search, LLM integration, and graph visualization.

10.1 Document Processing Pipeline

Upload (PDF/DOCX)

- > Save raw file to disk
- > Extract text (`pdfplumber` / `python-docx`)
- > Chunk text (heading-aware segmentation, max 500 chars)
- > Store metadata + chunks in SQLite
- > Index chunks in Qdrant vector store (via `mem0`)
- > Generate title/summary via DeepSeek LLM
- > Auto-assign to best-matching topic

10.2 Vector Storage

- **Engine:** Qdrant (local disk mode, no external server required).
- **Wrapper:** `mem0` library handles embedding generation and vector CRUD.
- **Embedding model:** `sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2` (384-dim, multilingual, supports Chinese).
- **Isolation:** vectors are partitioned by `user_id` (student ID).
- **Graceful degradation:** if the vector store fails to initialize (e.g., missing model cache), the system falls back to lexical search.

10.3 RAG (Retrieval-Augmented Generation)

1. User submits a question via `POST /knowledge/ask`.
2. System retrieves top-5 semantically similar chunks from the vector store.
3. Chunks are assembled into a context prompt.
4. The prompt is sent to DeepSeek Chat API (`deepseek-chat` model).
5. The LLM generates an answer grounded in the retrieved context.
6. Response includes both the answer and source chunk references.

10.4 Knowledge Graph

The graph is built on-demand via `GET /knowledge/graph`:

- **Nodes:** each chunk becomes a node, labeled by its heading.
- **Edges:** computed by pairwise semantic similarity (vector cosine) or lexical overlap as fallback. Only edges exceeding a configurable threshold are retained.
- **Topic routing:** if a query is provided, the system first identifies relevant topics via the topic vector store, then restricts the graph to chunks belonging to those topics.
- **Rendering:** the frontend uses Cytoscape.js with a force-directed (CoSE) layout.

10.5 Topic Tree

The knowledge tree provides hierarchical organization:

- Topics can be nested (parent/child relationships).
- Notes are assigned to topics (manually or automatically via vector similarity).
- Topic vectors are maintained in a separate Qdrant collection for query routing.
- Deleting a topic promotes its children to the parent level and reassigns notes.

11 Testing

The project maintains a comprehensive test suite with approximately 3,000 lines of test code across 16 test modules.

11.1 Test Categories

Table 8: Test modules and coverage areas

Module	Coverage
<code>test_hash_table.py</code>	Set/get/delete, collision handling, resize, iteration
<code>test_doubly_linked_list.py</code>	Append, prepend, insert, remove, find, iteration
<code>test_queue.py</code>	Enqueue, dequeue, empty queue behavior
<code>test_auth_service.py</code>	Registration, login, session validation, expiry
<code>test_schedule_service.py</code>	CRUD, week calculation, period detection, overview
<code>test_storage.py</code>	Atomic file I/O, user store, schedule store
<code>test_api.py</code>	Integration tests via FastAPI <code>TestClient</code>
<code>test_note_service.py</code>	Upload, text extraction, chunking logic
<code>test_note_store.py</code>	SQLite CRUD for notes and chunks
<code>test_knowledge_service.py</code>	Vector indexing, search, RAG, graph building
<code>test_knowledge_tree.py</code>	Topic CRUD, hierarchy, assignment
<code>test_knowledge_graph.py</code>	Graph construction, edge filtering, topic routing
<code>test_note_knowledge_integration.py</code>	End-to-end: upload → index → search → ask
<code>test_scnu_playwright.py</code>	Playwright-based scraper (mocked)
<code>test_browser_e2e.py</code>	Full browser E2E flow
<code>test_logging_config.py</code>	Log file rotation and formatting

11.2 Testing Strategy

- **Unit tests:** isolated testing of data structures and service methods with mocked dependencies.
- **Integration tests:** FastAPI `TestClient` for API endpoint testing with real storage (temporary directories).
- **Browser E2E:** Playwright-based tests that exercise the full user flow from login through knowledge workspace operations.
- **Fixtures:** shared `conf/test.py` provides temporary data directories, pre-configured services, and test users.

11.3 Running Tests

```
# Activate virtual environment
python_env\Scripts\activate

# Run all tests
python -m pytest tests/ -v
```

```
# Run specific module
python -m pytest tests/test_hash_table.py -v
```

12 Logging and Observability

The system uses Loguru for structured logging with multiple output sinks:

- **Combined daily log:** all messages at INFO level and above, rotated daily.
- **HTTP log:** request/response logs with method, path, status code, and duration.
- **Error log:** ERROR and above, for quick diagnosis.
- **Failure log:** CRITICAL level, for unrecoverable issues.

An HTTP middleware measures request duration and logs every request with its status code. Application lifecycle events (startup, shutdown, service initialization) are also logged.

13 Deployment and Configuration

13.1 Configuration

All configuration is centralized in `app/config.py`:

- File paths (data directory, logs, note files, Qdrant DB).
- Session parameters (cookie name, expiry, security flags).
- Class period timetable (12 periods with start/end times).
- SCNU portal URLs and paths.
- DeepSeek API credentials and model selection.
- Text chunking parameters (max length, overlap).
- HuggingFace cache directories (for offline model loading).

13.2 Running the Application

```
# Install dependencies
pip install -r requirements.txt

# Start the server
python -m uvicorn app.main:app --host 127.0.0.1 --port 8080

# Access the application
# Browser: http://127.0.0.1:8080/login
# API docs: http://127.0.0.1:8080/docs
```

13.3 Dependencies

Key Python packages (18 direct dependencies):

- | | |
|---------------------|--------------------|
| • fastapi | • requests |
| • uvicorn[standard] | • pdfplumber |
| • pydantic | • python-multipart |

- passlib[bcrypt]
- rsa
- playwright
- loguru
- httpx
- pytest
- python-docx
- mem0ai ($\geq 2.0.0$)
- qdrant-client
- sentence-transformers
- openai

14 Design Decisions and Trade-offs

1. **File-based storage over databases:** chosen to satisfy the course requirement of implementing custom data structures for indexing. JSON files are human-readable and easy to debug. SQLite is used only for the note module where relational queries are beneficial.
2. **In-memory session store:** sessions live in the custom hash table rather than a persistent store. This simplifies the implementation but means sessions are lost on server restart. Acceptable for a course project.
3. **Synchronous service layer with threaded tasks:** the SCNU scraper runs in daemon threads rather than using `asyncio` tasks. This avoids complexity while still providing non-blocking behavior for the user.
4. **Graceful degradation:** the knowledge module is designed to work even when external services (LLM, vector store) are unavailable. Search falls back to lexical matching; Q&A shows raw search results; graph uses token-overlap similarity.
5. **Monolithic deployment:** all components run in a single process. This is appropriate for the project scale and simplifies development/testing.
6. **Vue2 over a modern SPA framework:** chosen for simplicity and CDN availability. No build step is required; the frontend is plain HTML files with inline Vue templates.

15 Project Highlights

This section summarizes the key strengths that distinguish our project from a typical course assignment.

15.1 Solving a Real Problem with Real Data

Unlike many course projects that operate on synthetic or hard-coded data, our system is designed around a genuine, everyday student need: managing class schedules at SCNU. The entire data pipeline is built on real-world data sources:

- The schedule data comes directly from the university's live academic portal (`jwxt.scnu.edu.cn`), not from fabricated sample files.
- Users authenticate with their own university credentials to fetch their personal timetable—the system processes only the authenticated user's own data.
- The knowledge workspace operates on actual course notes uploaded by the student, producing real semantic search results and AI-generated answers.

This grounding in real data means the system must handle real-world messiness: inconsistent portal responses, network timeouts, varied PDF layouts, and multilingual text in notes.

15.2 Reverse Engineering and Web Scraping

Fetching schedule data required substantial reverse-engineering effort:

- **Login flow analysis:** the SCNU academic system uses a multi-step authentication process involving CSRF tokens and RSA-encrypted passwords. We reverse-engineered this flow by inspecting browser network traffic and reimplemented it in Python.
- **RSA password encryption:** the portal's login page dynamically fetches an RSA public key (modulus + exponent) and encrypts the password client-side before submission. Our scraper replicates this cryptographic step using the `rsa` library.
- **API discovery:** the schedule query endpoint, its parameters, and response format were identified through traffic analysis rather than official documentation.
- **Resilient fallback design:** recognizing that reverse-engineered APIs are inherently fragile, we implemented a three-tier fallback strategy (API $\rightarrow$ Playwright browser automation $\rightarrow$ PDF upload) to ensure the system remains functional even when the portal changes.

Compliance Statement. All reverse-engineering and scraping work in this project was carried out exclusively through legitimate, non-intrusive technical means:

- **Observable traffic only:** analysis was limited to HTTP request/response headers and bodies captured via standard browser developer tools during normal, authenticated use of the portal by the student's own account.
- **Frontend debugging only:** JavaScript logic (such as the client-side RSA encryption step) was understood by reading the portal's publicly delivered frontend source code in the browser debugger—exactly the same information any visitor receives.
- **No exploitation:** the project does *not* rely on, attempt to discover, or make use of any security vulnerability, bug exploit, privilege escalation, authentication bypass, or cryptographic break against the university or college systems.
- **No credentials of others:** the scraper only operates with credentials that the authenticated user voluntarily provides for their own account. It never attempts to access other users' data, enumerate accounts, or brute-force credentials.
- **Behaviorally equivalent to a human user:** from the portal's perspective, our requests are indistinguishable in content and intent from a student manually logging in and viewing their own schedule—only the client is automated. Requests are rate-limited (serialized through a queue) to avoid placing undue load on the portal.
- **Respectful fallbacks:** when the reverse-engineered API changes or is unavailable, the system falls back to Playwright (a legitimate browser driving the same public login flow) or, ultimately, to manual PDF upload by the user. At no point does the system attempt to defeat anti-bot measures through illicit means.

In short, every technique used here is standard web-development practice: traffic inspection, frontend debugging, and replaying well-formed HTTP requests. The project produces no tools, payloads, or techniques that could be used to compromise the portal, and all scraping is performed exclusively on the authenticated user's own account data.

15.3 Security Considerations

The system incorporates security practices appropriate for handling user credentials:

- **Password hashing:** local account passwords are never stored in plaintext. The system uses bcrypt (via passlib) or PBKDF2-SHA256 with 390,000 iterations as a fallback.
- **RSA credential encryption:** when transmitting university portal credentials to the backend for schedule fetching, the password is encrypted using the portal's RSA public key before network transmission, matching the security model of the portal itself.
- **Session token security:** session tokens are generated using `secrets.token_urlsafe(32)` (cryptographically secure), stored server-side only, and transmitted via `HttpOnly` cookies to prevent XSS-based token theft.
- **Backend-guarded routes:** protected pages are enforced at the server level (not merely hidden in the frontend). Unauthenticated requests receive HTTP redirects or 401 responses regardless of how the request is crafted.
- **No credential persistence:** university portal passwords are used transiently during the scraping task and are never written to disk or logged.

15.4 Frontend–Backend Separation

The system follows a clean frontend–backend separation:

- The backend exposes a well-defined RESTful JSON API (documented via Swagger/OpenAPI at `/docs`).
- The frontend communicates exclusively through HTTP endpoints—it has no direct access to storage, data structures, or business logic.
- API endpoints are implementation-agnostic: the frontend does not know (or need to know) whether data is stored in JSON files, SQLite, or a vector database.
- This separation enables independent development and testing of frontend and backend, and would allow replacing the Vue2 frontend with a mobile app or alternative UI without backend changes.

15.5 Note Knowledge Clustering and AI-Assisted Learning

A distinctive feature of this project is the integrated note knowledge workspace, which transforms static course notes into an actively queryable learning resource:

- **Fast topic-based organization:** students upload PDF or DOCX notes and either manually assign them to topics or let the system auto-route notes to the best-matching topic via vector similarity. This turns a flat pile of files into a structured, course-scoped knowledge tree with minimal manual effort.
- **Heading-aware automatic chunking:** the note service recognizes common heading styles (Markdown headers, Chinese chapter markers, numbered sections) and splits documents into semantically meaningful chunks rather than arbitrary byte windows, preserving the logical structure of the original note.
- **Initial RAG capability:** the system implements a basic but functional Retrieval-Augmented Generation pipeline. Questions are answered by first retrieving the top-5 semantically relevant chunks from the student's own notes via vector search, then prompting an LLM to synthesize an answer grounded in that retrieved context—not from the model's pretraining alone.

- **Cloud LLM integration for Q&A:** the system connects to the DeepSeek cloud API (OpenAI-compatible) to provide natural-language answering over uploaded notes. This gives students a personal tutor-like experience focused specifically on their own course material, at negligible per-query cost.
- **Source attribution:** every AI-generated answer is accompanied by the source chunks it was derived from, allowing students to verify the answer against the original material and discouraging blind trust in model output.
- **Knowledge graph visualization:** chunk-to-chunk similarities are computed and rendered as an interactive force-directed graph, helping students visually identify conceptual clusters and cross-references across their notes.
- **Offline-capable fallback:** when the cloud LLM or vector store is unavailable, the system gracefully degrades to lexical search and returns raw retrieved chunks, ensuring the workspace remains usable without internet connectivity or API credits.

Together, these capabilities turn the schedule system into a lightweight personal learning platform: students can see what class is happening now, jump directly into its knowledge workspace, upload the day's notes, and immediately search or ask questions across all material for that course.

15.6 Professional Version Control Practices

The project uses Git and GitHub following industry-standard workflows:

- **Branch strategy:** development occurs on feature branches (`feat/service`, `feat/front`), with `master` reserved for stable, reviewed code.
- **Pull Request workflow:** changes are merged via GitHub Pull Requests, enabling code review and discussion before integration.
- **Meaningful commit messages:** commits describe the intent of changes (e.g., “feat/service: Refactor logging system to improve testability”) rather than generic labels.
- **Gitignore discipline:** runtime data, virtual environments, caches, and credentials are excluded from version control.
- **Collaborative development:** multiple team members contribute to different modules concurrently, with conflicts resolved through standard merge workflows.

15.7 Production-Grade Engineering Practices

Beyond the course requirements, the project adopts several practices common in professional software development:

- **Structured logging:** Loguru-based logging with daily rotation, severity-based sinks, and request-level tracing—not just `print()` statements.
- **Atomic file operations:** JSON persistence uses write-to-temp-then-rename to prevent data corruption on crashes.
- **Thread safety:** all shared mutable state is protected by locks, even though the system is single-process.
- **Graceful degradation:** every external dependency (LLM, vector store, portal API) has a fallback path, ensuring the core system remains functional under partial failure.
- **Comprehensive test suite:** ~3,000 lines of tests covering unit, integration, and browser E2E scenarios—not just happy-path smoke tests.

- **Input validation:** Pydantic models enforce type constraints, length limits, and value ranges at the API boundary, preventing malformed data from reaching business logic.

16 Conclusion

This project demonstrates the practical application of fundamental data structures within a realistic web application context. The four hand-implemented structures (hash table, doubly linked list, BST, and queue) are not merely academic exercises but serve genuine roles in session management, course indexing, ordered traversal, and task scheduling.

What sets this project apart from a typical course assignment is its commitment to solving a real problem with real data. Rather than operating on fabricated inputs, the system reverse-engineers a live university portal, handles actual user credentials securely, and processes real course notes. This grounding in reality forced us to confront challenges—authentication protocols, API fragility, data inconsistency, security obligations—that purely academic projects never encounter.

Beyond the core OOP requirements, the system extends into modern software engineering territory with:

- A clean layered architecture with strict frontend-backend separation.
- RESTful API design with comprehensive endpoint coverage and auto-generated documentation.
- Reverse-engineered integration with the SCNU academic portal, with multi-tier fallbacks.
- Security-conscious credential handling (RSA encryption, bcrypt/PBKDF2 hashing, HttpOnly cookies).
- AI-powered features (vector search, RAG, knowledge graphs) with graceful degradation.
- A lightweight personal learning workspace: topic-based note clustering, initial RAG pipeline, and cloud-LLM-assisted Q&A grounded in the student's own notes.
- Professional version control via Git/GitHub with branching, PRs, and code review.
- A thorough test suite covering unit, integration, and E2E scenarios.
- Structured logging, atomic persistence, and thread-safe shared state.

The result is a functional prototype that a student could use daily to manage their schedule and course knowledge, while also serving as a demonstration of OOP principles, software architecture, and modern development practices.